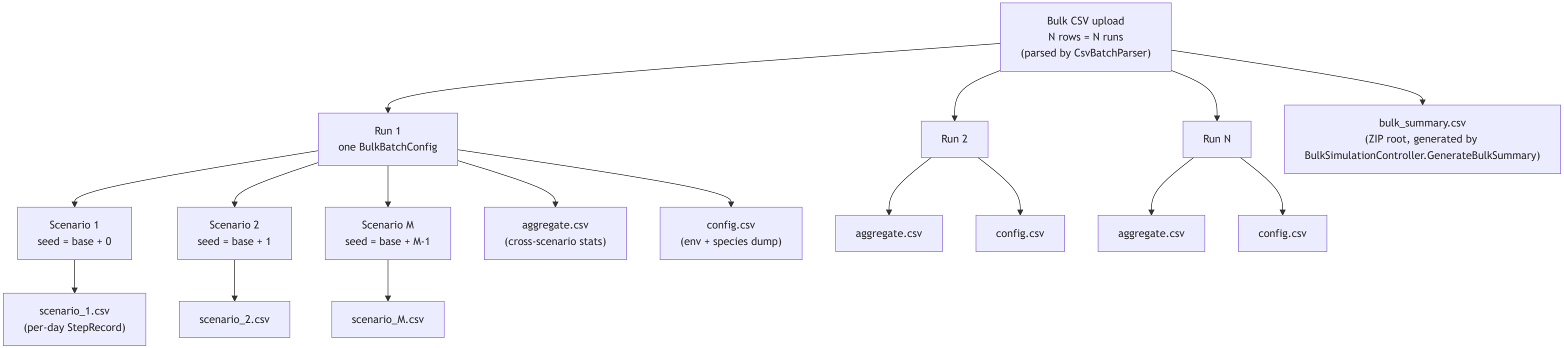
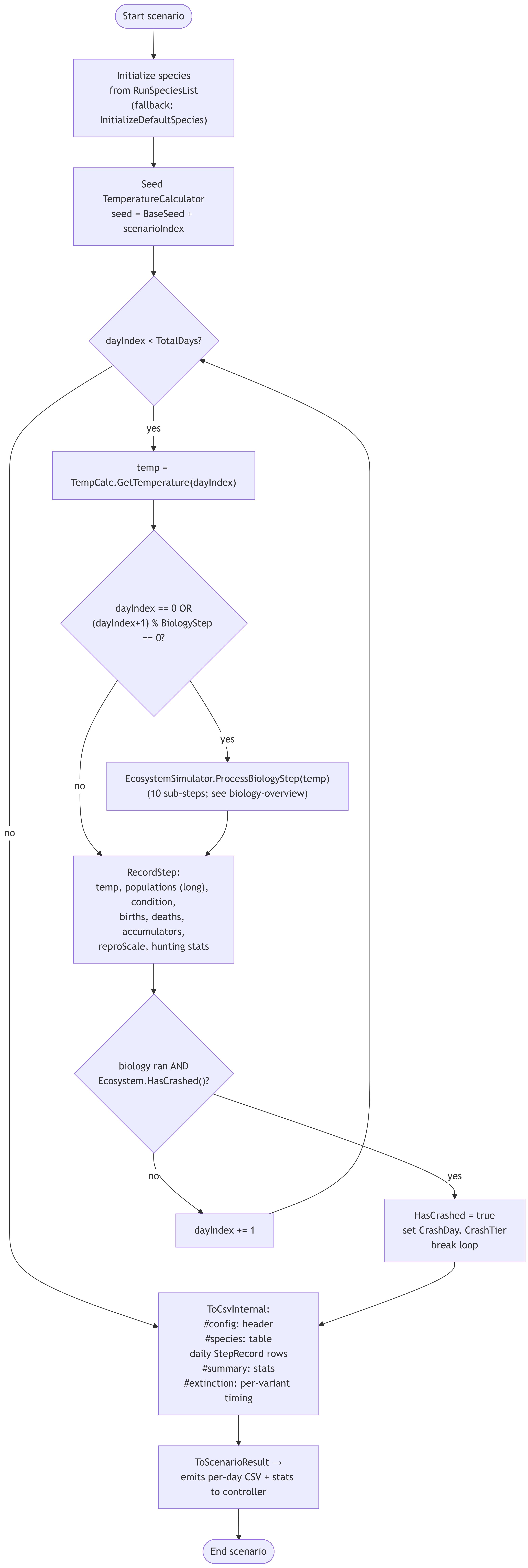
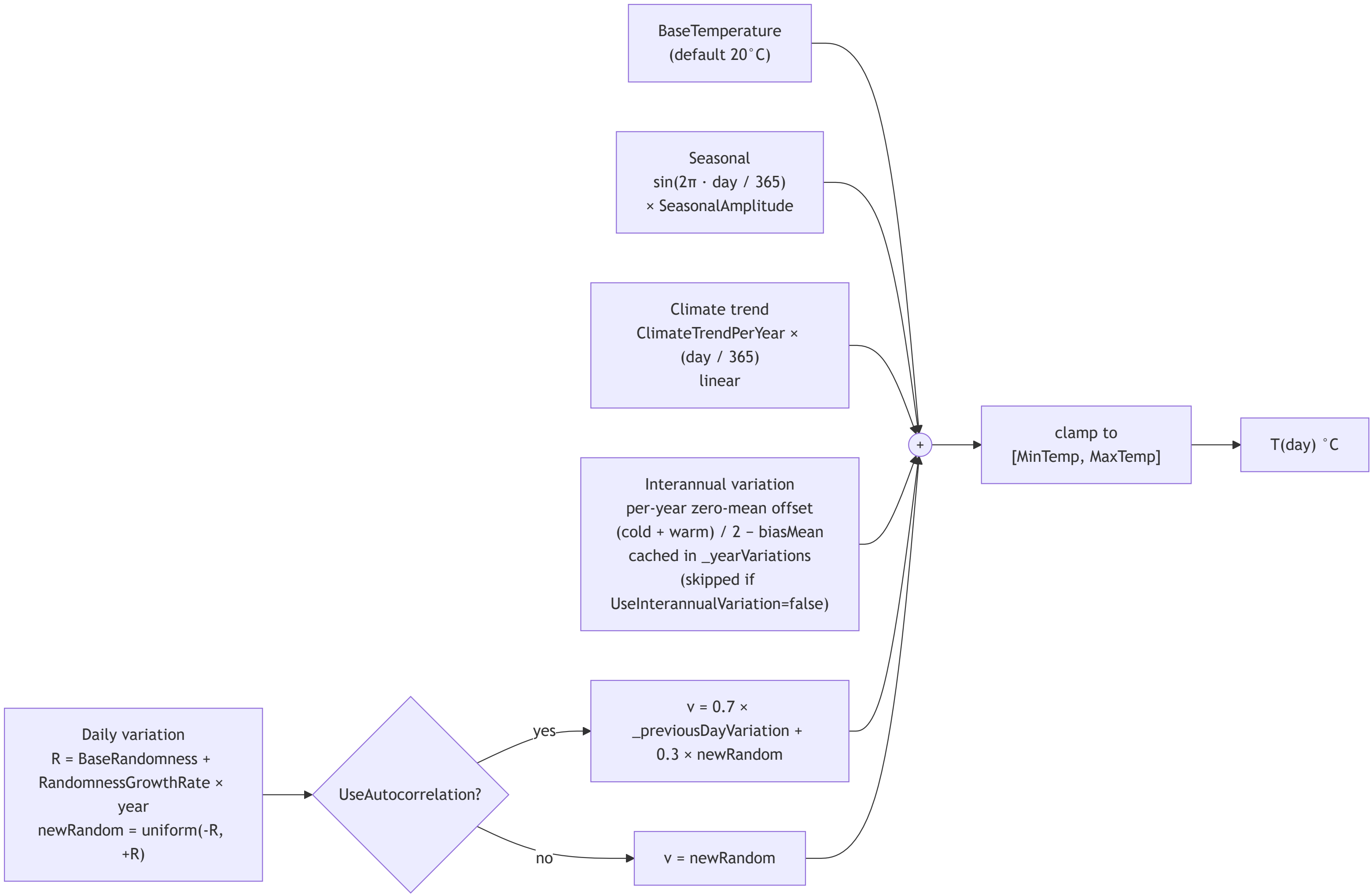
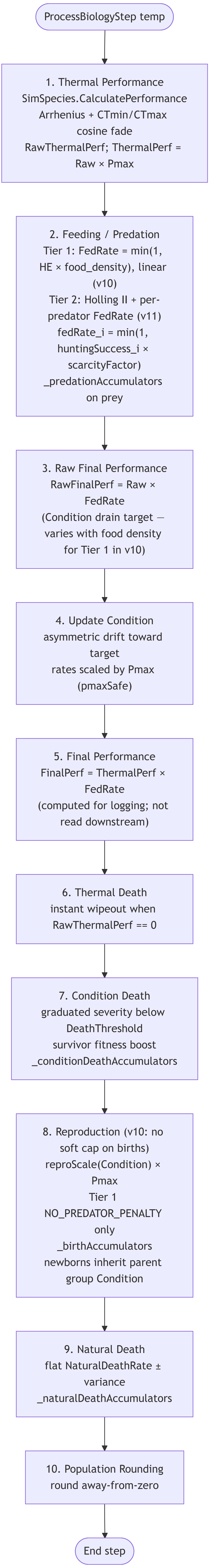
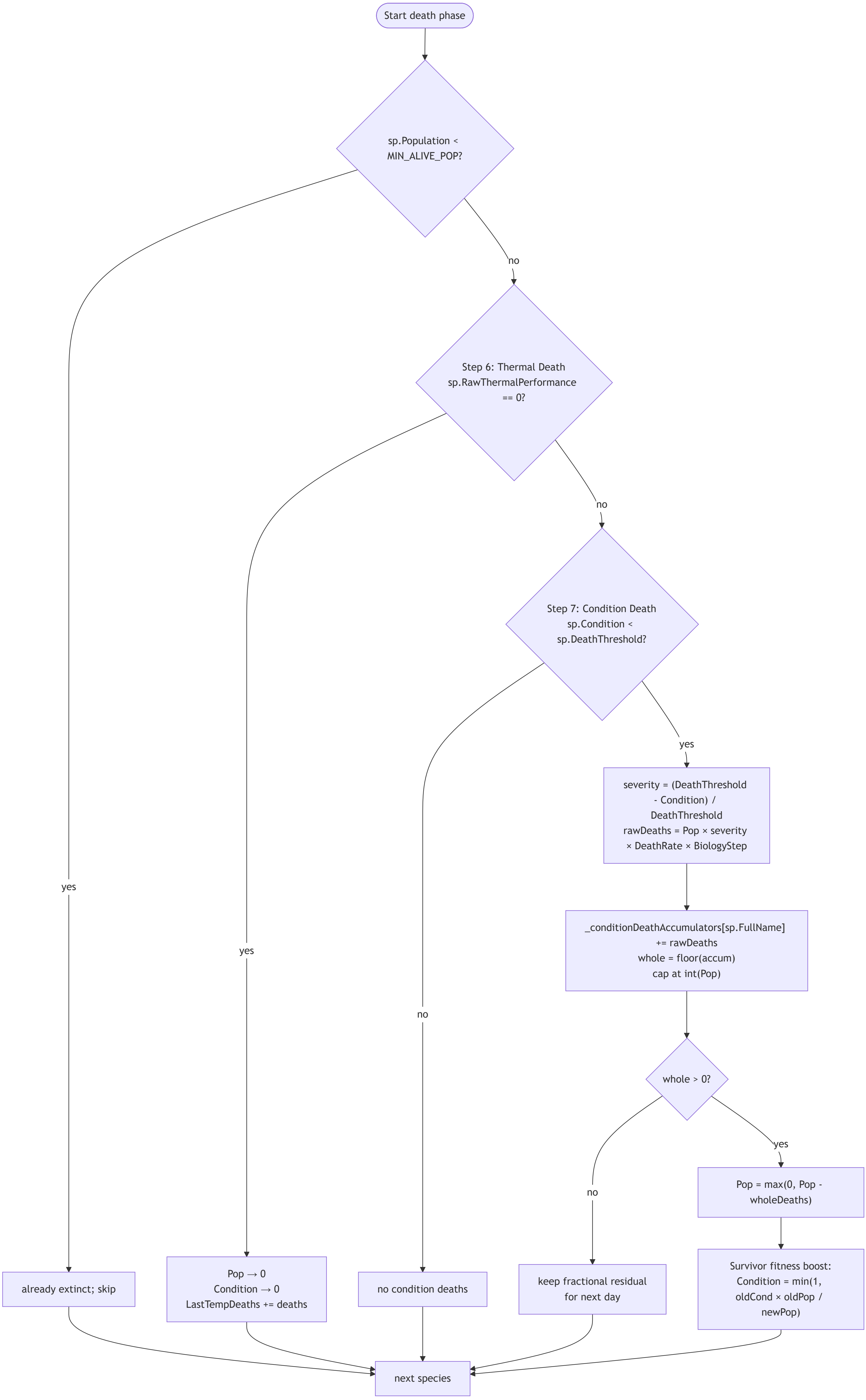

Bulk CSV upload
N rows = N runs
(parsed by CsvBatchParser)
Run 1
one BulkBatchConfig
Run 2
Run N
bulk_summary.csv
(ZIP root, generated by
BulkSimulationController.GenerateBulkSummary)
Scenario 1
seed = base + 0
Scenario 2
seed = base + 1
Scenario M
seed = base + M-1
aggregate.csv
(cross-scenario stats)
config.csv
(env + species dump)
aggregate.csv
config.csv
aggregate.csv
config.csv
scenario_1.csv
(per-day StepRecord)
scenario_2.csv
scenario_M.csv


Start scenario
Initialize species from RunSpeciesList (fallback: InitializeDefaultSpecies)
Seed TemperatureCalculator seed = BaseSeed + scenarioIndex
dayIndex < TotalDays?
yes
no
temp = TempCalc.GetTemperature(dayIndex)
dayIndex == 0 OR (dayIndex+1) % BiologyStep == 0?
yes
no
EcosystemSimulator.ProcessBiologyStep(temp) (10 sub-steps; see biology-overview)
RecordStep: temp, populations (long), condition, births, deaths, accumulators, reproScale, hunting stats
biology ran AND Ecosystem.HasCrashed()?
no
yes
dayIndex += 1
HasCrashed = true set CrashDay, CrashTier break loop
ToCsvInternal: #config: header #species: table daily StepRecord rows #summary: stats #extinction: per-variant timing
ToScenarioResult → emits per-day CSV + stats to controller
End scenario

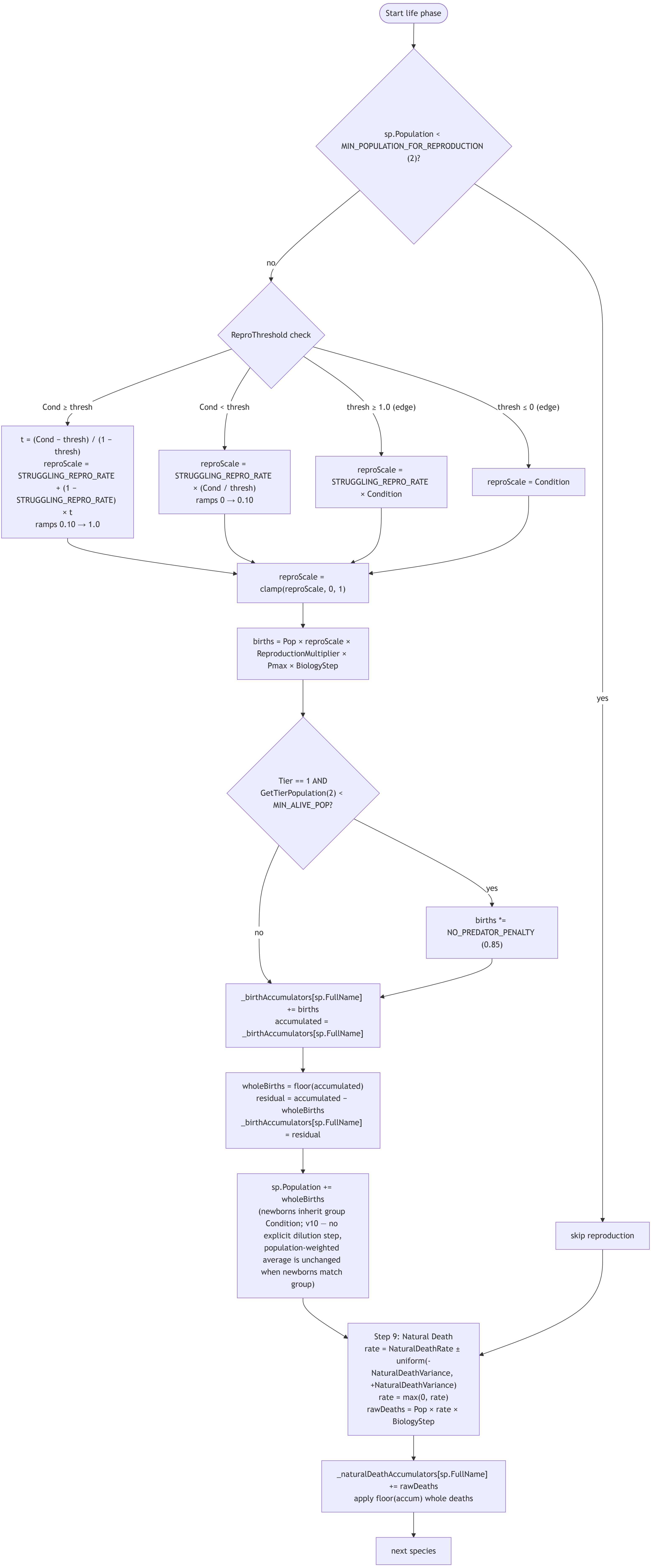

BaseTemperature
(default 20°C)
Seasonal
sin(2π · day / 365)
× SeasonalAmplitude
Climate trend
ClimateTrendPerYear ×
(day / 365)
linear
Interannual variation
per-year zero-mean offset
(cold + warm) / 2 − biasMean
cached in _yearVariations
(skipped if
UseInterannualVariation=false)
+
clamp to
[MinTemp, MaxTemp]
T(day) °C
Daily variation
R = BaseRandomness +
RandomnessGrowthRate ×
year
newRandom = uniform(-R,
+R)
UseAutocorrelation?
yes
no
v = 0.7 ×
_previousDayVariation +
0.3 × newRandom
v = newRandom


ProcessBiologyStep temp
1. Thermal Performance
SimSpecies.CalculatePerformance
Arrhenius + CTmin/CTmax
cosine fade
RawThermalPerf; ThermalPerf =
Raw × Pmax
2. Feeding / Predation
Tier 1: FedRate = min(1,
HE × food_density), linear
(v10)
Tier 2: Holling II + per-
predator FedRate (v11)
fedRate_i = min(1,
huntingSuccess_i ×
scarcityFactor)
_predationAccumulators
on prey
3. Raw Final Performance
RawFinalPerf = Raw ×
FedRate
(Condition drain target —
varies with food density
for Tier 1 in v10)
4. Update Condition
asymmetric drift toward
target
rates scaled by Pmax
(pmaxSafe)
5. Final Performance
FinalPerf = ThermalPerf ×
FedRate
(computed for logging; not
read downstream)
6. Thermal Death
instant wipeout when
RawThermalPerf == 0
7. Condition Death
graduated severity below
DeathThreshold
survivor fitness boost
_conditionDeathAccumulators
8. Reproduction (v10: no
soft cap on births)
reproScale(Condition) ×
Pmax
Tier 1
NO_PREDATOR_PENALTY
only
_birthAccumulators
newborns inherit parent
group Condition
9. Natural Death
flat NaturalDeathRate ±
variance
_naturalDeathAccumulators
10. Population Rounding
round away-from-zero
End step

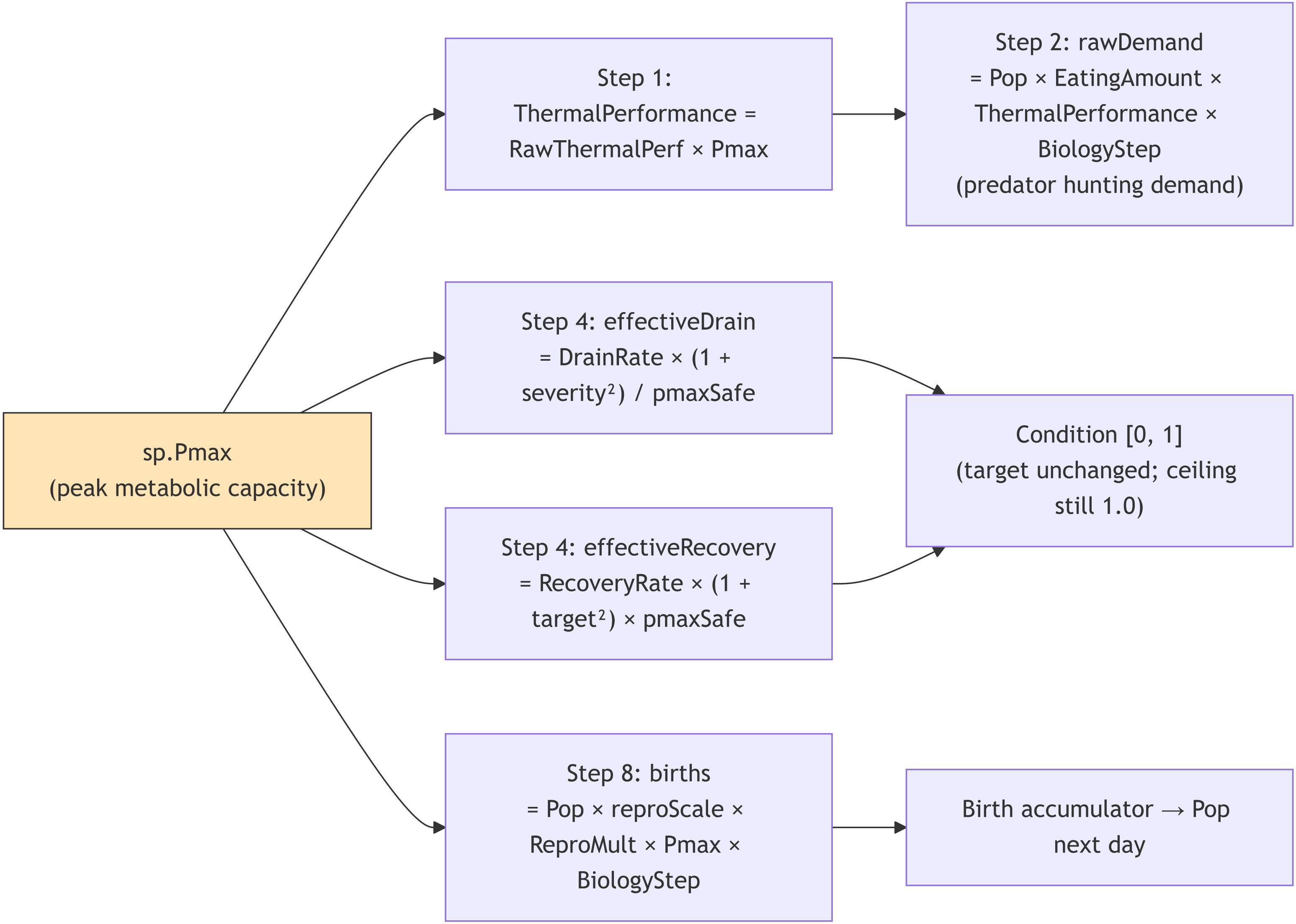

Temperature °C
T_eff = T + sp.TemperatureDebuff
Cosine fade over LETHAL_TRANSITION_WIDTH = 2°C near CTminC / CTmaxC
Arrhenius in Kelvin: numerator = exp(B/OT − B/T) · (1 + exp(L/OT − L/LB) + exp(U/UB − U/OT)) denominator = 1 + exp(L/T − L/LB) + exp(U/UB − U/T) perf = clamp(num/den, 0, 1)
RawThermalPerformance = perf × fade
ThermalPerformance = Raw × Pmax
Predator rawDemand = Pop × EatingAmount × ThermalPerf × BiologyStep
Holling II success holling = ratio / (ratio + halfSat) halfSat = NORMAL_PREY_RATIO · (1 − base) / base + variance in [-HuntingVariance, +HuntingVariance] clamp to [0, 1]
actualDemand = rawDemand × huntingSuccess
tier1Pop = sum of Tier 1 populations
totalEaten = min(availablePrey, Σ actualDemand)
food_density = max(0, 1 − tier1Pop / cap) cap always on (v11.1); cap floored at 1
scarcityFactor (v11) = totalEaten / totalActualDemand (1.0 when prey abundant)
FedRate (Tier 1, v10) = min(1, HE × food_density) linear extraction (NOT Holling II)
FedRate_i (Tier 2, v11) = min(1, huntingSuccess_i × scarcityFactor) (per-predator; LastFedRateT2 = pop-weighted avg)
RawFinalPerformance = Raw × FedRate (Condition drain target)
Condition vs target (pmaxSafe = max(Pmax, 1e-4))
Cond > target
Cond < target
severity = (1 - target)² effectiveDrain = ConditionDrainRate · (1 + severity) / pmaxSafe Condition -= (Condition - target) × effectiveDrain
boost = target² effectiveRecovery = ConditionRecoveryRate · (1 + boost) × pmaxSafe Condition += (target - Condition) × effectiveRecovery
Condition = clamp(Condition, 0, 1)
FinalPerformance = ThermalPerf × FedRate (logging only)

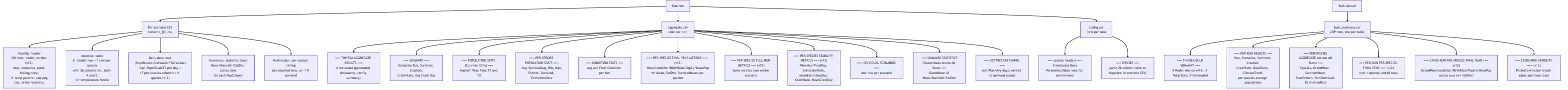


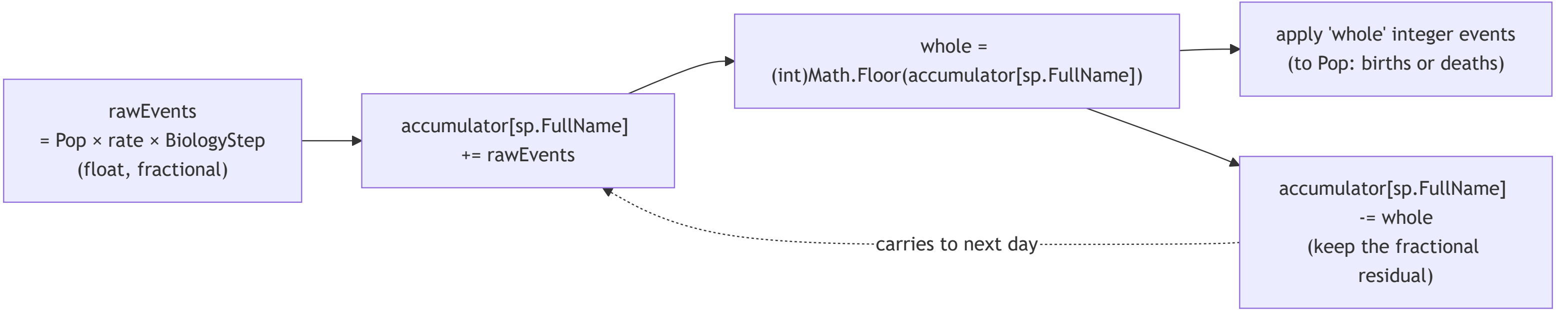
Start death phase
sp.Population < MIN_ALIVE_POP?
no
yes
Step 6: Thermal Death sp.RawThermalPerformance == 0?
no
yes
Step 7: Condition Death sp.Condition < sp.DeathThreshold?
yes
no
severity = (DeathThreshold - Condition) / DeathThreshold rawDeaths = Pop × severity × DeathRate × BiologyStep
_conditionDeathAccumulators[sp.FullName] += rawDeaths whole = floor(accum) cap at int(Pop)
whole > 0?
yes
no
Pop = max(0, Pop - wholeDeaths)
Survivor fitness boost: Condition = min(1, oldCond × oldPop / newPop)
already extinct; skip
Pop → 0 Condition → 0 LastTempDeaths += deaths
no condition deaths
keep fractional residual for next day
next species



Start life phase
sp.Population < MIN_POPULATION_FOR_REPRODUCTION (2)?
no
yes
ReproThreshold check
Cond ≥ thresh
Cond < thresh
thresh ≥ 1.0 (edge)
thresh ≤ 0 (edge)
t = (Cond − thresh) / (1 − thresh)
reproScale = STRUGGLING_REPRO_RATE + (1 − STRUGGLING_REPRO_RATE) × t
ramps 0.10 → 1.0
reproScale = STRUGGLING_REPRO_RATE × (Cond / thresh)
ramps 0 → 0.10
reproScale = STRUGGLING_REPRO_RATE × Condition
reproScale = Condition
reproScale = clamp(reproScale, 0, 1)
births = Pop × reproScale × ReproductionMultiplier × Pmax × BiologyStep
Tier == 1 AND GetTierPopulation(2) < MIN_ALIVE_POP?
yes
no
births *= NO_PREDATOR_PENALTY (0.85)
_birthAccumulators[sp.FullName] += births
accumulated = _birthAccumulators[sp.FullName]
wholeBirths = floor(accumulated)
residual = accumulated − wholeBirths
_birthAccumulators[sp.FullName] = residual
sp.Population += wholeBirths
(newborns inherit group Condition; v10 — no explicit dilution step, population-weighted average is unchanged when newborns match group)
skip reproduction
Step 9: Natural Death
rate = NaturalDeathRate ± uniform(-NaturalDeathVariance, +NaturalDeathVariance)
rate = max(0, rate)
rawDeaths = Pop × rate × BiologyStep
_naturalDeathAccumulators[sp.FullName] += rawDeaths
apply floor(accum) whole deaths
next species


sp.Pmax
(peak metabolic capacity)
Step 1:
ThermalPerformance =
RawThermalPerf × Pmax
Step 2: rawDemand
= Pop × EatingAmount ×
ThermalPerformance ×
BiologyStep
(predator hunting demand)
Step 4: effectiveDrain
= DrainRate × (1 +
severity²) / pmaxSafe
Step 4: effectiveRecovery
= RecoveryRate × (1 +
target²) × pmaxSafe
Condition [0, 1]
(target unchanged; ceiling
still 1.0)
Step 8: births
= Pop × reproScale ×
ReproMult × Pmax ×
BiologyStep
Birth accumulator → Pop
next day


One run
Per-scenario CSV
scenario_[N].csv
#config: header
(20 lines: model_version (v12),
days, scenarios, seed, biology step,
11 temp params, carrying cap, drain/recovery)
#species: table
(1 header row + 1 row per species
with 26 columns inc. both K and C
for temperature fields)
Daily data rows
(StepRecord.CsvHeader/ToCsvLine)
Day..ReproScaleT2 per day +
17 per-species columns × N species (v12)
#summary: statistics block
Mean/Max/Min/StdDev across days
for each PopColumn
#extinction: per-variant timing
day reached zero, or -1 if survived
aggregate.csv
(one per run)
=== TINYSEA AGGREGATE RESULTS ===
metadata (generated timestamp, config summary)
=== SUMMARY ===
Scenarios Run, Survived, Crashed,
Crash Rate, Avg Crash Day
=== POPULATION STATS (Survived Only) ===
Avg/Min/Max Final T1 and T2
=== PER-SPECIES POPULATION STATS ===
Avg, SurvivedAvg, Min, Max, Extinct, Survived, ExtinctionRate
=== CONDITION STATS ===
Avg and Final Condition per tier
=== PER-SPECIES FINAL YEAR METRICS === (v12)
MeanCondition/BirthRate/PopCv/MeanPop
w/ Mean, StdDev, SurvivedMean per species
=== PER-SPECIES FULL-RUN METRICS === (v12)
Same metrics over entire scenario
=== PER-SPECIES STABILITY METRICS === (v12)
Min/Max/FinalPop, ExtinctionRate, MeanExtinctionDay, CrashRate, MeanCrashDay
=== INDIVIDUAL SCENARIOS ===
one row per scenario
=== SUMMARY STATISTICS (Grand Mean Across All Runs) ===
GrandMean of Mean/Max/Min/StdDev
=== EXTINCTION TIMING ===
Min/Max/Avg days; extinct vs survived counts
config.csv
(one per run)
=== section headers ===
metadata lines
Parameter,Value rows for environment
=== SPECIES ===
(same 26-column table as #species: in scenario CSV)
Bulk upload
bulk_summary.csv
(ZIP root, one per bulk)
=== TINYSEA BULK SUMMARY ===
Model Version (v12), # Total Runs, # Generated
=== PER-RUN RESULTS ===
Run, Scenarios, Survived, Crashed,
CrashRate, BaseTemp, ClimateTrend,
per-species average populations
=== PER-SPECIES AGGREGATE (Across All Runs) ===
Species, GrandMean, SurvivedMean,
RunsExtinct, RunsSurvived, ExtinctionRate
=== PER-RUN PER-SPECIES FINAL YEAR === (v12)
(run × species) detail rows
=== CROSS-RUN PER-SPECIES FINAL YEAR === (v12)
GrandMeanCondition/BirthRate/PopCv/MeanPop
across runs (w/ StdDev)
=== CROSS-RUN STABILITY === (v12)
Pooled extinction/crash rates and mean days


rawEvents
= Pop × rate × BiologyStep
(float, fractional)
accumulator[sp.FullName]
+= rawEvents
whole =
(int)Math.Floor(accumulator[sp.FullName])
apply 'whole' integer events
(to Pop: births or deaths)
accumulator[sp.FullName]
-= whole
(keep the fractional
residual)
carries to next day